

Criterion A: Problem Specification



Criterion A (4 marks) is your project's foundation. It defines the problem you are solving, the user you are solving it for, and the specific, measurable success criteria your product must meet. A strong Criterion A clearly explains the 'what' and 'why' of your project. This guide shows you how to write clear, testable success criteria and avoid the common pitfall of being too vague.

Criterion A: How to Define Your Problem

Criterion A is where you lay the foundation for your entire project. It explains **what** problem you are solving, **why** it needs a solution, and **how** you will measure success.

Getting this section right is critical, as your **Success Criteria** will be used to guide your planning (Criterion B), development (Criterion D), and evaluation (Criterion E).

Recommended Word Count: ~300 words

Part 1: The Problem Scenario (The "What" and "Why")

This is a clear, concise description of the problem you are setting out to solve. It's not just "I am making a library app," but rather why such an app is needed.

Your problem scenario must: Clearly describe the problem and its measurable solution requirements. You no longer need a specific client; instead, your problem can come from the world around us, other fields of knowledge, or a current issue in computing.

When describing the problem, avoid simply stating the challenge; instead, articulate it clearly as a scenario, asking the following guiding questions to establish measurable requirements:

Guiding Question	Focus	Purpose in Documentation
What is the current situation?	The present context and current limitations (e.g., manual process, existing inefficient system).	Establishes the necessity for change.

Guiding Question	Focus	Purpose in Documentation
What causes the problem?	The specific source of the difficulty (e.g., calculation errors, lack of centralised data storage, security vulnerability).	Justifies the required functionalities (Criterion C).
Who is affected by the problem?	The stakeholders or users (e.g., students, library staff, small business owner).	Defines user roles and interface needs.
What are the objectives/general requirements?	The high-level goals for the computational solution (e.g., must allow permanent record storage, must provide accurate statistical analysis).	Links directly to success criteria.

Part 2: The Computational Context (The "How")

In this section, you explain *what* kind of solution you will build and *why* you've chosen that path.

You need to justify your choice of:

- **Platform/Language:** Why are you building this in Python with a local database instead of as a web application? Why did you choose Java for an OOP-heavy solution?
- **Solution Type:** Explain your choice of a "stand-alone solution," "mobile application," "web-based solution," or "database solution."

You must clearly explain the choice of computational context for your solution. This justification links the nature of the problem to the technical environment you select (e.g., web-based, database, or stand-alone application).

Context Choice	Justification Rationale
Stand-alone Application (Python/Java)	Chosen because the solution needs to run quickly and efficiently for a single user without constant internet access, and the focus is on complex algorithms and/or object-oriented design.
Web-based Solution (HTML, CSS, JavaScript)	Chosen because the solution requires universal, platform-independent access for multiple simultaneous users (e.g., a multi-user inventory system or student dashboard).
Database Solution (SQL)	Chosen because the core problem involves the persistent storage, organisation, querying, and manipulation of relational data requiring integrity and retrieval speed.

Tip: Always try to go beyond generic justification by providing details relevant to your project. You can learn more in the [Extended Examples for Criterion A Documentation](#).

Part 3: The Success Criteria (The "Proof")

This is the most important part of Criterion A. You must create a list of **8-10 specific, measurable, and testable** outcomes that your product must achieve to be considered successful. These are derived directly from the problem's requirements.

A good success criterion is a clear "pass/fail" statement that you can prove with a test.

Good vs. Bad Success Criteria

Vague or subjective criteria cannot be properly tested and will lose you marks in Criteria C, D and E.

Bad Criteria (Vague / Subjective / Untestable)	Good Criteria (Specific / Measurable / Testable)
"The program will be user-friendly."	"The solution will be menu-driven with clearly named choices, including 1. adding books, 2. removing books, 3. registering patrons, 4. checking books in, 5. checking books out, 6. viewing overdue loans, 7. generating summary reports."
"The code will run without crashing."	"The solution will give appropriate warning messages for invalid data inputs (e.g., letters in a 'quantity' field)."
"The program will be aesthetically pleasing."	"The UI will include a visual dashboard displaying the number of overdue books."
"The code will use loops." (This is a technique, not a goal)	"The solution will allow the user to sort the inventory alphabetically by name and numerically by stock level."
"The program will store data."	"The solution will permanently store all customer records (Name, ID, Email) in the 'Customers' database table."
	"The solution will allow for the input, editing, and deletion of all customer records from the database."

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**